```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv("/content/Mental Health Dataset.csv", header=0)
df.head()
```

	Timestamp	Gender	Country	Occupation	self_employed	family_history	treatment	Days_Indoors	Growing_Stress	Changes_Habi
0	8/27/2014 11:29	Female	United States	Corporate	NaN	No	Yes	1-14 days	Yes	
1	8/27/2014 11:31	Female	United States	Corporate	NaN	Yes	Yes	1-14 days	Yes	
2	8/27/2014 11:32	Female	United States	Corporate	NaN	Yes	Yes	1-14 days	Yes	
3	8/27/2014 11:37	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	
4	8/27/2014 11:43	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	

```
df.shape
```

```
(292364, 17)
```

```
df.describe()
```

	Timestamp	Gender	Country	Occupation	self_employed	family_history	treatment	Days_Indoors	Growing_Stress	Changes
count	292364	292364	292364	292364	287162	292364	292364	292364	292364	
unique	580	2	35	5	2	2	2	5	3	
top	8/27/2014 11:43	Male	United States	Housewife	No	No	Yes	1-14 days	Maybe	
freq	2384	239850	171308	66351	257994	176832	147606	63548	99985	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 292364 entries, 0 to 292363
Data columns (total 17 columns):
 #   Column                   Non-Null Count   Dtype
---  ------                   --------------   -----
 0   Timestamp                292364 non-null  object
 1   Gender                   292364 non-null  object
 2   Country                  292364 non-null  object
 3   Occupation               292364 non-null  object
 4   self_employed            287162 non-null  object
 5   family_history           292364 non-null  object
 6   treatment                292364 non-null  object
 7   Days_Indoors             292364 non-null  object
 8   Growing_Stress           292364 non-null  object
 9   Changes_Habits           292364 non-null  object
 10  Mental_Health_History    292364 non-null  object
 11  Mood_Swings              292364 non-null  object
 12  Coping_Struggles         292364 non-null  object
 13  Work_Interest            292364 non-null  object
 14  Social_Weakness          292364 non-null  object
 15  mental_health_interview  292364 non-null  object
 16  care_options             292364 non-null  object
dtypes: object(17)
memory usage: 37.9+ MB
```

```
df.isnull().sum()
```

	0
Timestamp	0
Gender	0
Country	0
Occupation	0
self_employed	5202
family_history	0
treatment	0
Days_Indoors	0
Growing_Stress	0
Changes_Habits	0
Mental_Health_History	0
Mood_Swings	0
Coping_Struggles	0
Work_Interest	0
Social_Weakness	0
mental_health_interview	0
care_options	0

dtype: int64

```
df = df.drop(columns=["Timestamp"])
df["self_employed"] = df["self_employed"].fillna("No")
```

```
df
```

	Gender	Country	Occupation	self_employed	family_history	treatment	Days_Indoors	Growing_Stress	Changes_Habits	Me
0	Female	United States	Corporate	No	No	Yes	1-14 days	Yes	No	
1	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	No	
2	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	No	
3	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	No	
4	Female	United States	Corporate	No	Yes	Yes	1-14 days	Yes	No	
...	...	...	...	...	...	...	...	...	...	
292359	Male	United States	Business	Yes	Yes	Yes	15-30 days	No	Maybe	
292360	Male	South Africa	Business	No	Yes	Yes	15-30 days	No	Maybe	
292361	Male	United States	Business	No	Yes	No	15-30 days	No	Maybe	
292362	Male	United States	Business	No	Yes	Yes	15-30 days	No	Maybe	
292363	Male	United States	Business	No	Yes	Yes	15-30 days	No	Maybe	

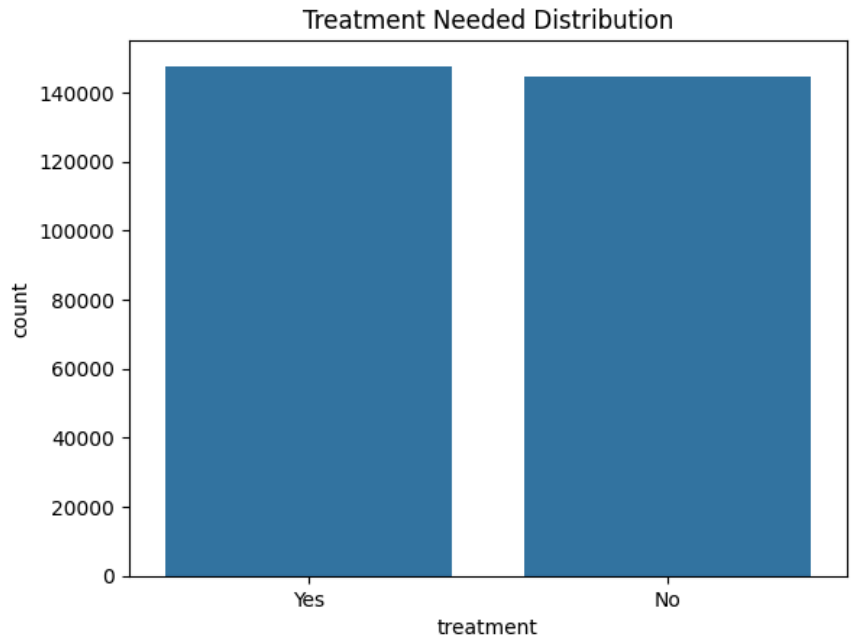
292364 rows × 16 columns

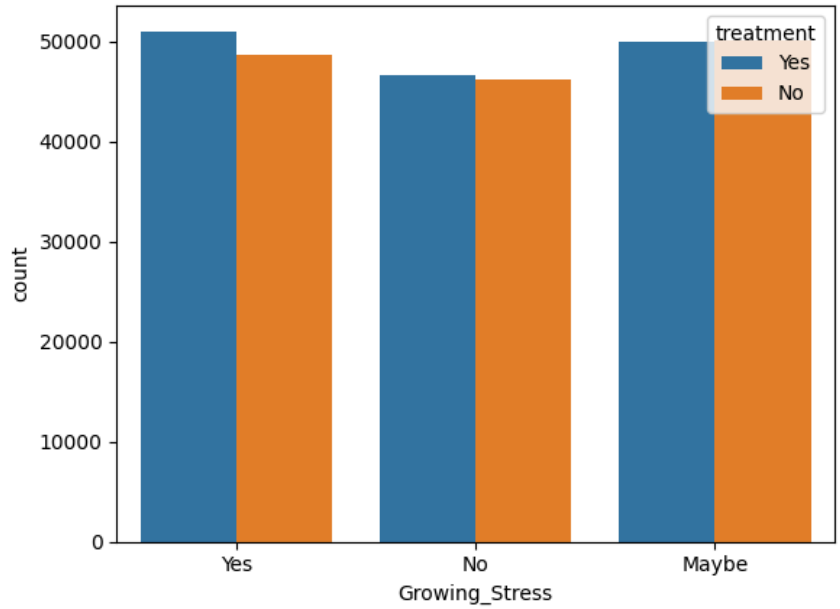
```
df.isnull().sum()
```

	0
Gender	0
Country	0
Occupation	0
self_employed	0
family_history	0
treatment	0
Days_Indoors	0
Growing_Stress	0
Changes_Habits	0
Mental_Health_History	0
Mood_Swings	0
Coping_Struggles	0
Work_Interest	0
Social_Weakness	0
mental_health_interview	0
care_options	0

dtype: int64

```
# Target balance
sns.countplot(x="treatment", data=df)
plt.title("Treatment Needed Distribution")
plt.show()

# Example: stress vs treatment
sns.countplot(x="Growing_Stress", hue="treatment", data=df)
plt.show()
```






```
target = "treatment"
X = df.drop(columns=[target])
y = df[target]
```

```
from sklearn.preprocessing import LabelEncoder

encoders = {}
X_encoded = X.copy()
for col in X_encoded.columns:
    le = LabelEncoder()
    X_encoded[col] = le.fit_transform(X_encoded[col])
    encoders[col] = le

y_le = LabelEncoder()
y_encoded = y_le.fit_transform(y)   # Yes=1, No=0 (check with y_le.classes_)
```

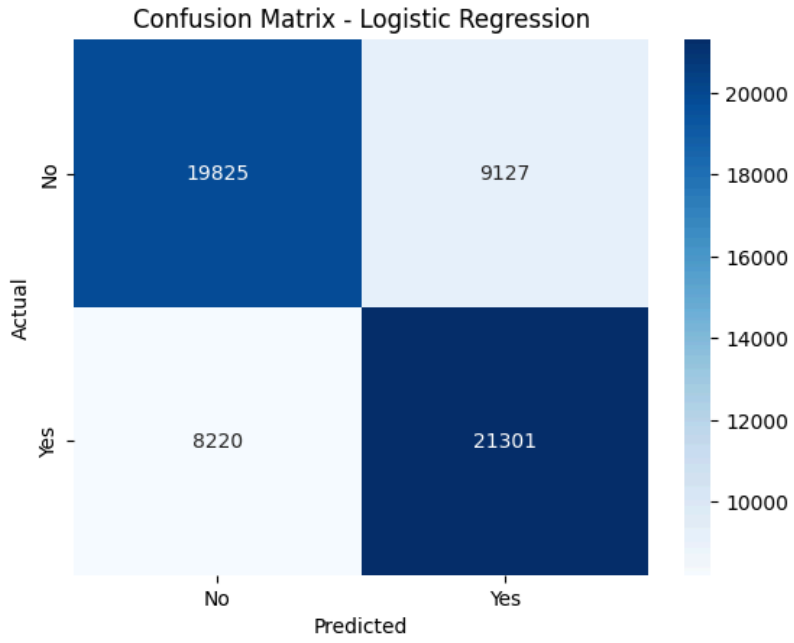
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
print(X_train.shape, X_test.shape)
```

```
(233891, 15) (58473, 15)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
```

```
log_preds_train = log_model.predict(X_train)
log_preds = log_model.predict(X_test)

log_train_acc = accuracy_score(y_train, log_preds_train)
log_acc = accuracy_score(y_test, log_preds)

print("Logistic Regression Training Accuracy:", log_train_acc)
print("Logistic Regression Test Accuracy:", log_acc)
print("\nClassification Report:\n", classification_report(y_test, log_preds))
tn, fp, fn, tp = cm.ravel()
print(f"True Negatives: {tn}, False Positives: {fp}")
print(f"False Negatives: {fn}, True Positives: {tp}")
print(f"\nFalse Negative Rate: {fn/(fn+tp):.2%}  (people who needed help but model said No)")

cm = confusion_matrix(y_test, log_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=y_le.classes_, yticklabels=y_le.classes_)
plt.title("Confusion Matrix - Logistic Regression")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

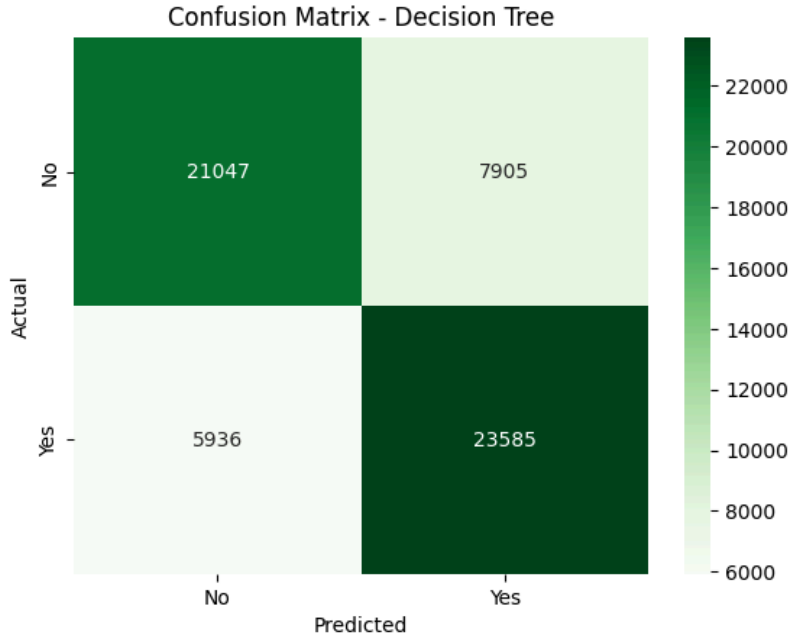
```
Logistic Regression Training Accuracy: 0.7040501772193303
Logistic Regression Test Accuracy: 0.7033331623142305

Classification Report:
              precision    recall  f1-score   support

           0       0.71      0.68      0.70     28952
           1       0.70      0.72      0.71     29521

    accuracy                           0.70     58473
   macro avg       0.70      0.70      0.70     58473
weighted avg       0.70      0.70      0.70     58473

True Negatives: 21047, False Positives: 7905
False Negatives: 5936, True Positives: 23585

False Negative Rate: 20.11%  (people who needed help but model said No)
```




```
from sklearn.tree import DecisionTreeClassifier, plot_tree

dt_model = DecisionTreeClassifier(max_depth=8, random_state=42)
dt_model.fit(X_train, y_train)

dt_preds_train = dt_model.predict(X_train)
dt_preds = dt_model.predict(X_test)

dt_train_acc = accuracy_score(y_train, dt_preds_train)
dt_acc = accuracy_score(y_test, dt_preds)

print("Decision Tree Training Accuracy:", dt_train_acc)
print("Decision Tree Test Accuracy:", dt_acc)
print("\nClassification Report:\n", classification_report(y_test, dt_preds))
tn, fp, fn, tp = cm.ravel()
print(f"True Negatives: {tn}, False Positives: {fp}")
print(f"False Negatives: {fn}, True Positives: {tp}")
print(f"\nFalse Negative Rate: {fn/(fn+tp):.2%}  (people who needed help but model said No)")
```

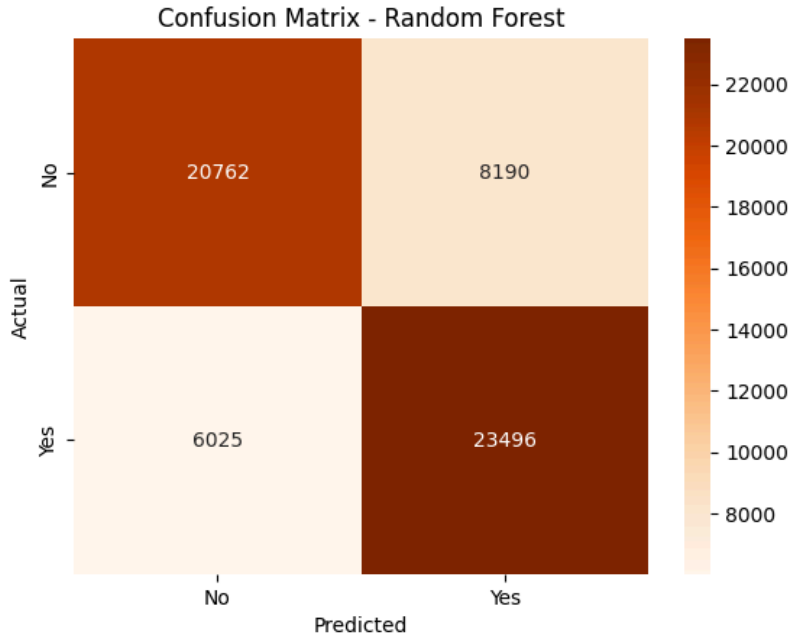
```
cm = confusion_matrix(y_test, dt_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
            xticklabels=y_le.classes_, yticklabels=y_le.classes_)
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
Decision Tree Training Accuracy: 0.7665707530430842
Decision Tree Test Accuracy: 0.7632924597677561

Classification Report:
               precision    recall  f1-score   support

           0       0.78      0.73      0.75     28952
           1       0.75      0.80      0.77     29521

    accuracy                           0.76     58473
   macro avg       0.76      0.76      0.76     58473
weighted avg       0.76      0.76      0.76     58473

True Negatives: 20762, False Positives: 8190
False Negatives: 6025, True Positives: 23496

False Negative Rate: 20.41%  (people who needed help but model said No)
```



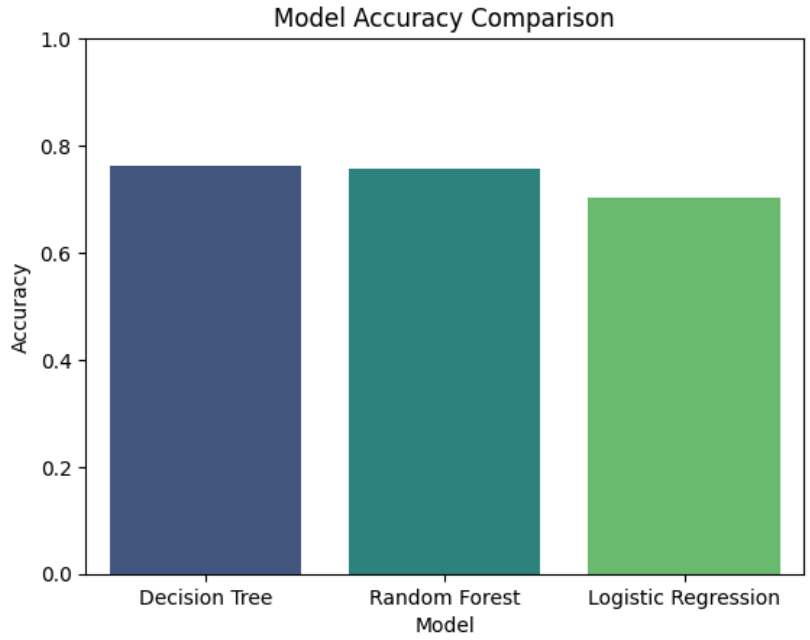

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=200, max_depth=12, random_state=42, n_jobs=-1)
rf_model.fit(X_train, y_train)

rf_preds_train = rf_model.predict(X_train)
rf_preds = rf_model.predict(X_test)

rf_train_acc = accuracy_score(y_train, rf_preds_train)
rf_acc = accuracy_score(y_test, rf_preds)

print("Random Forest Training Accuracy:", rf_train_acc)
print("Random Forest Test Accuracy:", rf_acc)
print("\nClassification Report:\n", classification_report(y_test, rf_preds))
tn, fp, fn, tp = cm.ravel()
print(f"True Negatives: {tn}, False Positives: {fp}")
print(f"False Negatives: {fn}, True Positives: {tp}")
print(f"\nFalse Negative Rate: {fn/(fn+tp):.2%}  (people who needed help but model said No)")

cm = confusion_matrix(y_test, rf_preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Oranges",
            xticklabels=y_le.classes_, yticklabels=y_le.classes_)
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
Random Forest Training Accuracy: 0.7636933443356093
Random Forest Test Accuracy: 0.7568963453217724

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.72      0.74     28952
           1       0.74      0.80      0.77     29521

    accuracy                           0.76     58473
   macro avg       0.76      0.76      0.76     58473
weighted avg       0.76      0.76      0.76     58473

True Negatives: 20762, False Positives: 8190
False Negatives: 6025, True Positives: 23496

False Negative Rate: 20.41%  (people who needed help but model said No)
```




```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [log_acc, dt_acc, rf_acc]
}).sort_values("Accuracy", ascending=False)

print(comparison)

sns.barplot(x="Model", y="Accuracy", data=comparison, hue="Model", palette="viridis", legend=False)
plt.ylim(0, 1)
plt.title("Model Accuracy Comparison")
plt.show()

best_model_name = comparison.iloc[0]["Model"]
best_model = {"Logistic Regression": log_model, "Decision Tree": dt_model, "Random Forest": rf_model}[best_model_name]
print("\nBest model:", best_model_name)
```

```
                Model  Accuracy
1       Decision Tree  0.763292
2       Random Forest  0.756896
0  Logistic Regression  0.703333
```




```
Best model: Decision Tree
```

```
if best_model_name == "Random Forest":
    importances = pd.Series(best_model.feature_importances_, index=X_encoded.columns)
    importances.sort_values().plot(kind="barh", figsize=(8,6))
    plt.title("Feature Importance")
    plt.show()
```

Save the Model

```
import joblib

joblib.dump(best_model, "mental_health_model.pkl")
joblib.dump(encoders, "feature_encoders.pkl")
joblib.dump(y_le, "target_encoder.pkl")
joblib.dump(list(X_encoded.columns), "feature_columns.pkl")

print("Saved: mental_health_model.pkl, feature_encoders.pkl, target_encoder.pkl, feature_columns.pkl")
```

```
Saved: mental_health_model.pkl, feature_encoders.pkl, target_encoder.pkl, feature_columns.pkl
```

NLP

```
import nltk
nltk.download('vader_lexicon')
from nltk.sentiment import SentimentIntensityAnalyzer

sia = SentimentIntensityAnalyzer()

def analyze_journal_entry(text):
    scores = sia.polarity_scores(text)
    compound = scores['compound']
    if compound <= -0.5:
        mood = "Highly Negative"
    elif compound <= -0.1:
        mood = "Negative"
    elif compound < 0.1:
        mood = "Neutral"
    elif compound < 0.5:
        mood = "Positive"
    else:
        mood = "Highly Positive"
    return mood, scores

# test
```

```
print(analyze_journal_entry("I feel so exhausted and hopeless lately, nothing feels worth it."))
print(analyze_journal_entry("I had a great day, feeling productive and calm."))
```

```
('Highly Negative', {'neg': 0.525, 'neu': 0.475, 'pos': 0.0, 'compound': -0.7732})
('Highly Positive', {'neg': 0.0, 'neu': 0.336, 'pos': 0.664, 'compound': 0.7845})
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
```

```
def combined_risk_assessment(ml_prediction, journal_mood):
    """
    ml_prediction: 'Yes'/'No' from the questionnaire model
    journal_mood: output of analyze_journal_entry
    """
    risk = "Low"
    if ml_prediction == "Yes" and journal_mood in ["Negative", "Highly Negative"]:
        risk = "High"
    elif ml_prediction == "Yes" or journal_mood in ["Negative", "Highly Negative"]:
        risk = "Moderate"
    return risk
```

```
!pip install -q streamlit
!pip install -q nltk
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.3/10.3 MB 73.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.4/11.4 MB 80.2 MB/s eta 0:00:00
```

Streamlit

```
%%writefile app.py
import streamlit as st
import pandas as pd
import joblib
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer

@st.cache_resource
def load_sentiment_analyzer():
    try:
        nltk.data.find('sentiment/vader_lexicon.zip')
    except LookupError:
        nltk.download('vader_lexicon')
    return SentimentIntensityAnalyzer()

sia = load_sentiment_analyzer()

@st.cache_resource
def load_ml_artifacts():
    model = joblib.load("mental_health_model.pkl")
    encoders = joblib.load("feature_encoders.pkl")
    target_encoder = joblib.load("target_encoder.pkl")
    feature_columns = joblib.load("feature_columns.pkl")
    return model, encoders, target_encoder, feature_columns

model, encoders, target_encoder, feature_columns = load_ml_artifacts()

st.set_page_config(page_title="AI Mental Health Monitor", page_icon="🧠", layout="centered")
st.title("🧠 AI-Based Mental Health Monitoring")
st.write(
    "This tool combines a survey-based ML risk model with NLP sentiment analysis "
    "of your own words. It is **not a diagnostic tool** — please seek professional "
    "help if you're struggling."
)

tab1, tab2, tab3 = st.tabs(["📋 Questionnaire (ML)", "📝 Journal Entry (NLP)", "📊 Combined Result"])

def safe_encode(col, value):
    le = encoders[col]
    if value not in le.classes_:
        return 0
    return le.transform([value])[0]

def analyze_journal_entry(text):
    scores = sia.polarity_scores(text)
    compound = scores['compound']
    if compound <= -0.5:
        mood = "Highly Negative"
    elif compound <= -0.1:
        mood = "Negative"
    elif compound < 0.1:
```

```
        mood = "Neutral"
    elif compound < 0.5:
        mood = "Positive"
    else:
        mood = "Highly Positive"
    return mood, scores

def combined_risk_assessment(ml_pred, journal_mood):
    if ml_pred == "Yes" and journal_mood in ["Negative", "Highly Negative"]:
        return "High"
    elif ml_pred == "Yes" or journal_mood in ["Negative", "Highly Negative"]:
        return "Moderate"
    return "Low"

with tab1:
    st.subheader("Answer a few questions")
    gender = st.selectbox("Gender", ["Male", "Female"])
    country = st.text_input("Country", "United States")
    occupation = st.selectbox("Occupation", ["Corporate", "Student", "Business", "Housewife", "Others"])
    self_employed = st.selectbox("Self-employed?", ["Yes", "No"])
    family_history = st.selectbox("Family history of mental illness?", ["Yes", "No"])
    days_indoors = st.selectbox("Days spent indoors recently",
                                ["Go out Every day", "1-14 days", "15-30 days", "31-60 days", "More than 2 months"])
    growing_stress = st.selectbox("Do you feel growing stress?", ["Yes", "No", "Maybe"])
    changes_habits = st.selectbox("Noticed changes in habits?", ["Yes", "No", "Maybe"])
    mh_history = st.selectbox("Personal mental health history?", ["Yes", "No", "Maybe"])
    mood_swings = st.selectbox("Mood swings", ["Low", "Medium", "High"])
    coping_struggles = st.selectbox("Struggling to cope?", ["Yes", "No"])
    work_interest = st.selectbox("Lost interest in work?", ["Yes", "No", "Maybe"])
    social_weakness = st.selectbox("Feeling social weakness/withdrawal?", ["Yes", "No", "Maybe"])
    mh_interview = st.selectbox("Comfortable discussing mental health in an interview?", ["Yes", "No", "Maybe"])
    care_options = st.selectbox("Aware of care options at work/school?", ["Yes", "No", "Not sure"])

    if st.button("Predict Risk from Questionnaire"):
        raw_input = {
            "Gender": gender, "Country": country, "Occupation": occupation
```